

Secure File Sharing System Report

Names: Simugomwa Obald

Date: 06-01-2026

1. Introduction

This document provides a detailed security overview of the Secure File Sharing System developed as part of a hands-on cybersecurity internship project task 03. The system allows users to securely upload and download files while ensuring confidentiality, integrity, and controlled access through encryption and password-based protection.

The primary security goal of this project is to ensure that files remain protected both at rest and during access, even if storage is compromised.

2. Security Objectives

The system was designed with the following security objectives:

- Ensure uploaded files are never stored in plaintext
- Enforce password-based access control per file
- Prevent unauthorized file downloads
- Protect encryption keys from exposure
- Follow basic cryptographic best practices suitable for real-world applications
-

3. Encryption Methodology

3.1 Encryption Algorithm

The system uses the Advanced Encryption Standard (AES) with the following configuration:

Parameter	Value
-----------	-------

Algorithm	AES
-----------	-----

Key Size	256-bit
----------	---------

Mode	CBC (Cipher Block Chaining)
------	-----------------------------

Padding	PKCS7
---------	-------

AES-256 is an industry-standard symmetric encryption algorithm widely used in secure systems such as VPNs, cloud storage, and enterprise applications.

3.2 Why AES-CBC?

- AES provides strong confidentiality
- CBC mode ensures that identical plaintext blocks produce different ciphertext
- A random Initialization Vector (IV) prevents pattern analysis
- PKCS7 padding ensures compatibility with block-size requirements

4. File Upload (Encryption Flow)

1. User selects a file and creates a password
2. A cryptographically secure random salt is generated
3. The password and salt are passed to PBKDF2 to derive an AES-256 key
4. A random Initialization Vector (IV) is generated
5. The file is encrypted using AES-CBC
6. The encrypted file is stored on the server
7. Only encrypted data is written to disk

4.2 File Download (Decryption Flow)

1. User requests a file download
2. A password prompt (modal) is displayed
3. The provided password is validated
4. If valid, the AES key is re-derived using the stored salt
5. The encrypted file is decrypted in memory
6. The original file is sent to the user for download

If the password is incorrect, decryption is denied and no file data is leaked.

5. Metadata Storage

Metadata includes:

- Filename

- Password hash
- Base64-encoded salt

Example structure:

```
{  
  "example.pdf": {  
    "password_hash": "hashed_value",  
    "salt": "encoded_salt"  
  }  
}
```

6.Key Management Strategy

Key Characteristics

- Encryption keys are **derived dynamically**
- Keys are never stored on disk
- Keys exist only in memory during encryption or decryption
- Each file has a **unique derived key**

Sensitive configuration values (e.g., system secrets) are stored using **environment variables**, not hardcoded in source code. This follows secure deployment practices and prevents accidental exposure through version control.

Data Security at Rest

- All stored files are encrypted using AES-256
- Storage compromise does not expose readable data
- Encrypted files appear as random binary data
- Unauthorized access to storage alone is insufficient to retrieve files

Access Control & User Interaction Security

- Password is required for every download attempt
- Incorrect passwords result in access denial
- Password entry is handled through a secure modal interface

- No password is passed via URL parameters

Conclusion

This secure file sharing system demonstrates the practical application of cryptographic principles, secure key handling, and user-focused security design. By combining AES encryption, password-based key derivation, and safe UI interactions, the system successfully protects sensitive data while remaining usable.

The project reflects real-world secure development practices commonly used in healthcare, legal, and enterprise environments.